03 — Dictionaries and Sets (beginner-friendly)

Two new containers. You already know **lists**; these two solve problems lists are bad at. Take this file slowly — section 2 (counting) is the single most useful thing in it.

---

1. What is a dictionary? (the labelled box)

A **list** stores things by **position** — item 0, item 1, item 2. That works when the order matters and you know where things are.

But what if you want to store **"ama is 20 years old"**? With a list you'd have to remember that position 3 happens to be Ama's age. That's fragile.

A **dictionary** stores each value under a **label you choose** — called the **key**:

```
ages = {"ama": 20, "kofi": 25}
```

Read that as: *"the label `"ama"` points to `20`, and the label `"kofi"` points to `25`."*

Real-world picture: a school register. You don't look up "the 4th student" — you look up "*Ama*" and find her mark. The name is the key; the mark is the value.

	List	Dictionary
You look things up by	position (`0, 1, 2`)	a label you chose (`"ama"`)
Written with	`[ ]` square brackets	`{ }` curly braces
Example	`[20, 25]`	`{"ama": 20, "kofi": 25}`

Using a dictionary

```
ages = {"ama": 20, "kofi": 25}

ages["ama"]          # 20  — look up by key (like nums[0], but with
ages["yaw"] = 30     # add a NEW key with a value
ages["ama"] = 21     # same syntax OVERWRITES an existing key
"kofi" in ages       # True — "does this KEY exist?"
len(ages)            # 3 — how many keys
del ages["kofi"]     # remove that key
```

> **Note:** a key can appear **only once**. Assigning to an existing key replaces its value — it doesn't add a second copy. That's exactly what makes counting work (section 2).

Looping over a dictionary

```
for key in ages:                    # gives you each KEY
    print(key)                      # "ama", "yaw"

for key, value in ages.items():     # gives you BOTH at once
    print(key, value)               # "ama" 21 , "yaw" 30
```

Use `.items()` whenever you need the value too — it's the dictionary version of `enumerate`.

The `.get()` lifesaver

Asking for a key that doesn't exist **crashes your program**:

```
ages["zoe"]            # KeyError — CRASH
ages.get("zoe", 0)     # 0  — returns your chosen default instead o
```

Read `ages.get("zoe", 0)` as: *"give me zoe's value, **or 0** if zoe isn't there."*

This matters enormously for counting, because the **first** time you meet something, it isn't in the dictionary yet.

---

2. The counting pattern — the most important thing in this file

The problem it solves: "how many times does each thing appear?"

Say you want to count letters in `"aba"`. By hand, you'd keep a tally sheet:

- see a → not on my sheet yet → write "a: 1"
- see b → not on my sheet yet → write "b: 1"
- see a → already there with 1 → cross it out, write "a: 2"

The dictionary IS the tally sheet. Here's that in code:

```
def count_items(items):
    counts = {}                              # the empty tally sheet
    for x in items:
        counts[x] = counts.get(x, 0) + 1     # old count (or 0) + 1,
    return counts
```

The one line to understand — read it right-to-left:

```
counts[x]  =  counts.get(x, 0)  +  1
   ↑             ↑                ↑
 store it     how many so far?   one more
              (0 if brand new)
```

Full trace for `["a", "b", "a"]`:

step	x	`counts.get(x, 0)`	`+ 1` → stored	tally sheet after
1	"a"	0 (never seen)	1	`{"a": 1}`
2	"b"	0 (never seen)	1	`{"a": 1, "b": 1}`
3	"a"	1 (seen once)	2	`{"a": 2, "b": 1}`

Result: `{"a": 2, "b": 1}` ✓

> **Why `.get(x, 0)` and not `counts[x]`?** On step 1, `"a"` isn't in the dictionary yet, so `counts["a"]` would crash. `.get(x, 0)` says "treat a missing key as 0" — which is exactly right for a tally starting from nothing.

The shortcut: `Counter`

Python has this pattern built in:

```
from collections import Counter

counts = Counter("mississippi")   # {'i':4, 's':4, 'p':2, 'm':1}
counts.most_common(1)             # [('i', 4)] — the most frequent,
counts["s"]                       # 4 — and missing keys give 0, no
```

Both are allowed on the exam. **Know both:** `Counter` is faster to type, but the manual version is what you need when you're storing something other than a plain count (like a position, in exercise 2).

---

3. What is a set? (the guest list)

A **set** is a bag of items where:

- **duplicates are impossible** — adding something twice changes nothing
- **there's no order** — no positions, no `set[0]`
- **checking "is X in here?" is instant**

Real-world picture: a party guest list. You only care about *"is this person on the list?"* — not what number they are, and their name appears only once.

```
seen = set()            # empty set  (NOT {} — that makes an empty d
seen.add(5)             # put 5 in
seen.add(5)             # adding again does nothing — still just one
5 in seen               # True — the instant check
len(seen)               # 1
```

Turning a list into a set **removes duplicates**, which is a handy trick on its own:

```
nums = [1, 2, 2, 3]
set(nums)               # {1, 2, 3}
len(set(nums))          # 3 — how many DISTINCT values
```

Combining sets

```
set(a) & set(b)         # items in BOTH        (think AND)
set(a) | set(b)         # items in EITHER      (think OR)
set(a) - set(b)         # in a but NOT in b
```

Why sets matter (the speed thing)

This is the whole reason sets exist for Q4.

When you write `x in my_list`, Python checks the items **one by one** from the start. For a list of 100,000 items that's up to 100,000 checks — **every single time**. Put that inside a loop over 100,000 items and you get 10,000,000,000 steps. Your program **times out** and scores zero.

`x in my_set` doesn't search at all — it jumps straight to the answer, no matter how big the set is.

> **The rule:** if you're checking "have I seen this before?" **inside a loop**, use a set, not a list. Swapping one for the other is often the only change needed to make a slow solution pass.

---

4. Worked examples (real GCA problems)

firstDuplicate — the first value that appears twice

The idea in one sentence: *walk the list, remember everything you pass, and stop the moment you meet something you already remember.*

```
def first_duplicate(nums):
    seen = set()              # everything I've walked past
    for x in nums:
        if x in seen:         # met this one before?
            return x          # yes → it's the answer, stop
        seen.add(x)           # no → remember it and keep going
    return -1                 # finished the list, never repeated
```

Trace `[2, 3, 3, 1]`:

x	is x in seen?	action	seen after
2	no	remember it	`{2}`
3	no	remember it	`{2, 3}`
3	**yes**	**return 3** ✓	—

Note this is a **search loop** — `return` inside the loop is correct here, because you stop as soon as you find it.

firstNotRepeatingCharacter — first character appearing exactly once

The idea: you can't know if a character repeats until you've seen the whole string. So: **count everything first, then walk again** and pick the first one whose count is 1.

```
from collections import Counter

def first_not_repeating(s):
    counts = Counter(s)       # PASS 1: count every character
    for c in s:               # PASS 2: walk in original order
        if counts[c] == 1:
            return c
    return '_'
```

For `"abca"`: counts are `{a:2, b:1, c:1}`. Walking again: a has 2 (skip), b has 1 → return `"b"`. The second loop walks the **original string** — that's what makes it the *first* one.

Two passes over the same data — completely normal, same as `all_longest_strings`.

Two Sum — do two numbers add up to a target?

The idea: for each number x, the partner it needs is `target - x`. Instead of hunting for that partner, **remember every number you pass** and just ask "did my partner already go by?"

```
def two_sum(nums, target):
    seen = set()
    for x in nums:
        if target - x in seen:    # has my partner already come by?
            return True
        seen.add(x)
    return False
```

Trace `nums = [3, 7]`, `target = 10`:

x	partner needed (`10 - x`)	is partner in seen?	seen after
3	7	no	{3}
7	3	**yes** → return True ✓	—

This "have I already seen what I need?" idea unlocks a lot of Q4 problems.

Grouping — put related things in buckets

The idea: make a **label** that's identical for things belonging together, then use a dictionary of lists — key = the label, value = the list of things with it.

For anagrams, sorting the letters makes a perfect label: `"eat"` and `"tea"` both sort to `"aet"`.

```
def group_anagrams(words):
    groups = {}
    for w in words:
        key = ''.join(sorted(w))              # "eat" → "aet"
        groups.setdefault(key, []).append(w)  # new key? start an e
    return list(groups.values())
```

`setdefault(key, [])` means "*give me the list at this key, creating an empty one if it's not there yet*" — then `.append(w)` puts the word in that bucket.

---

5. Which tool? — phrase → tool

The problem says...	Reach for
"how many times", "most frequent", "count each"	dict / `Counter`
"duplicate", "appears twice", "unique", "distinct"	set
"pair that sums to", "find its partner"	set (Two Sum idea)
"group by", "put the same ___ together"	dict of lists
"have we seen this before?"	set

6. Common mistakes with these two

- **`{}` is an empty DICTIONARY, not a set.** For an empty set write `set()`.
- **`d[key]` crashes on a missing key.** Use `d.get(key, default)` when it might be missing.
- **Sets have no order and no positions.** `my_set[0]` is an error.
- **`.add()` is for sets, `.append()` is for lists.** Both return `None` — call them, never assign them (see 10-my-common-mistakes.md).
- **Counting needs the count-then-decide split.** If a problem depends on totals, count in one pass and decide in a second — don't try to do both at once.

Exercises (in `practice/q4_problem_solving.py`)

1. `most_frequent(nums)` — the value that appears most often.
2. `contains_close_duplicates(nums, k)` — is there a duplicate at index distance ≤ k?
3. `common_elements(a, b)` — values in both lists, no duplicates, sorted.
4. `can_form_note(note, magazine)` — can you build `note` from `magazine`'s letters?
5. `longest_unique_substring(s)` — longest stretch with no repeated character. (Hardest — needs the sliding window from file 05.)

Which tool for each? 1 → `Counter` + `.most_common`. 2 → dict storing `value → last index`, compare the gap. 3 → `sorted(set(a) & set(b))`. 4 → `Counter` both, check each needed letter is available. 5 → set + sliding window (file 05).